混凝土抗压强度数据Regression Analysis

实验目的

本实验利用线性回归模型方法对混凝土抗压强度进行预测，通过对数据进行预处理、相关性分析、主成分分析以及线性回归建模，最后对比了原始特征模型与PCA降维模型的性能差异

数据加载与分析

数据集包含1030条样本，共9个变量，包括：

输入特征（8个）：cement、slag、flyash、water、superplasticizer、coarseaggregate、fineaggregate、age

目标变量（1个）：csMPa（抗压强度）

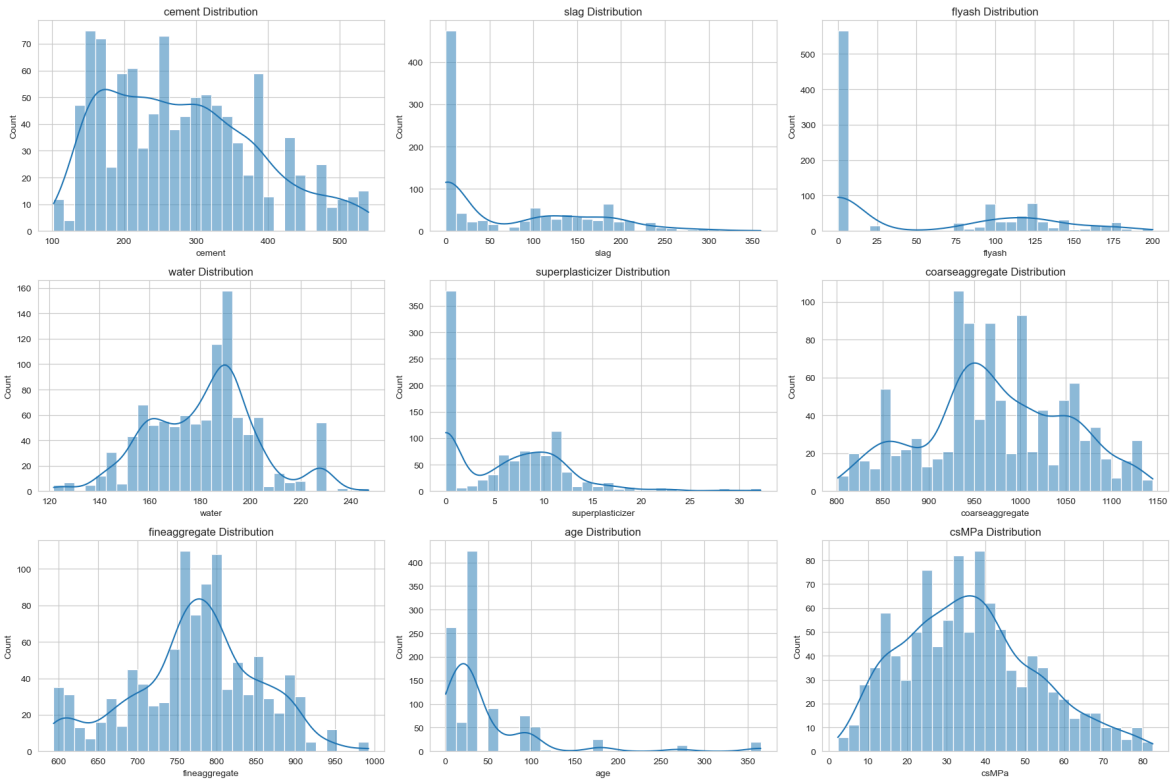
数据集中不存在缺失值，因此无需进行缺失值处理，保证了后续分析的完整性

In [1]:
```
import pandas as pd

# 导入数据，查看基本情况
data = pd.read_csv(r'Concrete_Data_Yeh.csv')
print("数据维度:", data.shape)  # 显示数据的维度
print("数据基本信息:\n",data.info())  # 显示数据的基本信息
print("数据前5行:\n",data.head())  # 显示数据的前几行
print("缺失值统计:\n",data.isnull().sum())  # 缺失值统计
print("数据描述统计:\n",data.describe().T)  # 显示数据的描述统计信息
```

```
数据维度: (1030, 9)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1030 entries, 0 to 1029
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   cement            1030 non-null   float64
 1   slag              1030 non-null   float64
 2   flyash            1030 non-null   float64
 3   water             1030 non-null   float64
 4   superplasticizer  1030 non-null   float64
 5   coarseaggregate   1030 non-null   float64
 6   fineaggregate     1030 non-null   float64
 7   age               1030 non-null   int64
 8   csMPa             1030 non-null   float64
dtypes: float64(8), int64(1)
memory usage: 72.5 KB
数据基本信息:
 None
数据前5行:
    cement   slag  flyash  water  superplasticizer  coarseaggregate  \
0   540.0    0.0     0.0  162.0               2.5           1040.0
1   540.0    0.0     0.0  162.0               2.5           1055.0
2   332.5  142.5     0.0  228.0               0.0            932.0
3   332.5  142.5     0.0  228.0               0.0            932.0
4   198.6  132.4     0.0  192.0               0.0            978.4

   fineaggregate  age  csMPa
0          676.0   28  79.99
1          676.0   28  61.89
2          594.0  270  40.27
3          594.0  365  41.05
4          825.5  360  44.30
缺失值统计:
 cement              0
slag                0
flyash              0
water               0
superplasticizer    0
coarseaggregate     0
fineaggregate       0
age                 0
csMPa               0
dtype: int64
数据描述统计:
                    count        mean         std     min      25%      50%  \
cement            1030.0  281.167864  104.506364  102.00  192.375  272.900
slag              1030.0   73.895825   86.279342    0.00    0.000   22.000
flyash            1030.0   54.188350   63.997004    0.00    0.000    0.000
water             1030.0  181.567282   21.354219  121.80  164.900  185.000
superplasticizer  1030.0    6.204660    5.973841    0.00    0.000    6.400
coarseaggregate   1030.0  972.918932   77.753954  801.00  932.000  968.000
fineaggregate     1030.0  773.580485   80.175980  594.00  730.950  779.500
age               1030.0   45.662136   63.169912    1.00    7.000   28.000
csMPa             1030.0   35.817961   16.705742    2.33   23.710   34.445

                       75%     max
cement             350.000   540.0
slag               142.950   359.4
flyash             118.300   200.1
```

```
water              192.000   247.0
superplasticizer    10.200    32.2
coarseaggregate   1029.400  1145.0
fineaggregate      824.000   992.6
age                 56.000   365.0
csMPa               46.135    82.6
```

In [ ]:
```
import seaborn as sns
import matplotlib.pyplot as plt

def plot_all_distributions(data):
    sns.set_style("whitegrid")
    columns = data.columns
    fig, axes = plt.subplots(3, 3, figsize=(18, 12))
    axes = axes.flatten()
    for i, col in enumerate(columns):
        sns.histplot(data=data,x=col,kde=True,bins=30,ax=axes[i])
        axes[i].set_title(f"{col} Distribution", fontsize=12)
        axes[i].set_xlabel(col, fontsize=10)
        axes[i].set_ylabel("Count", fontsize=10)

    plt.tight_layout()
    plt.show()

plot_all_distributions(data)
```

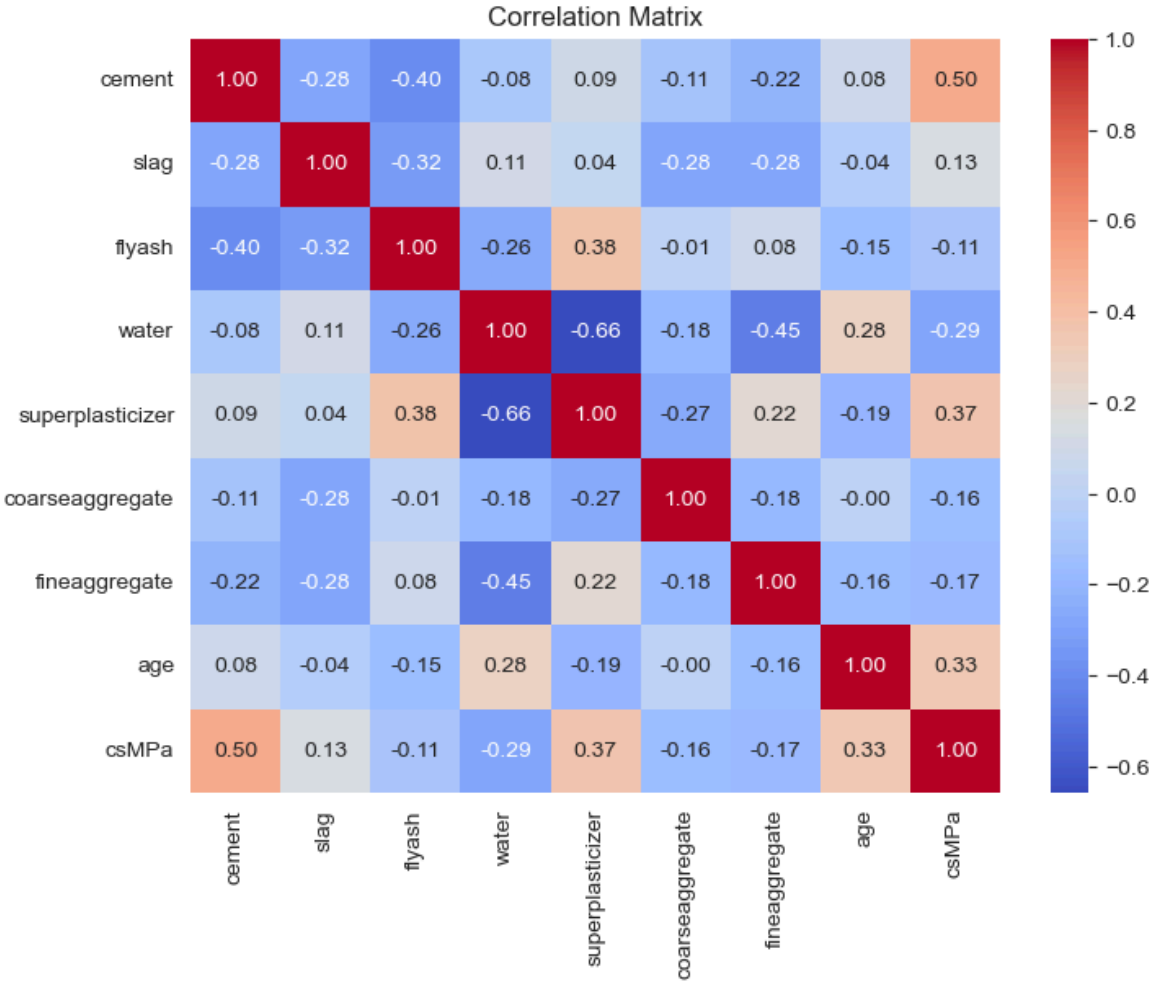


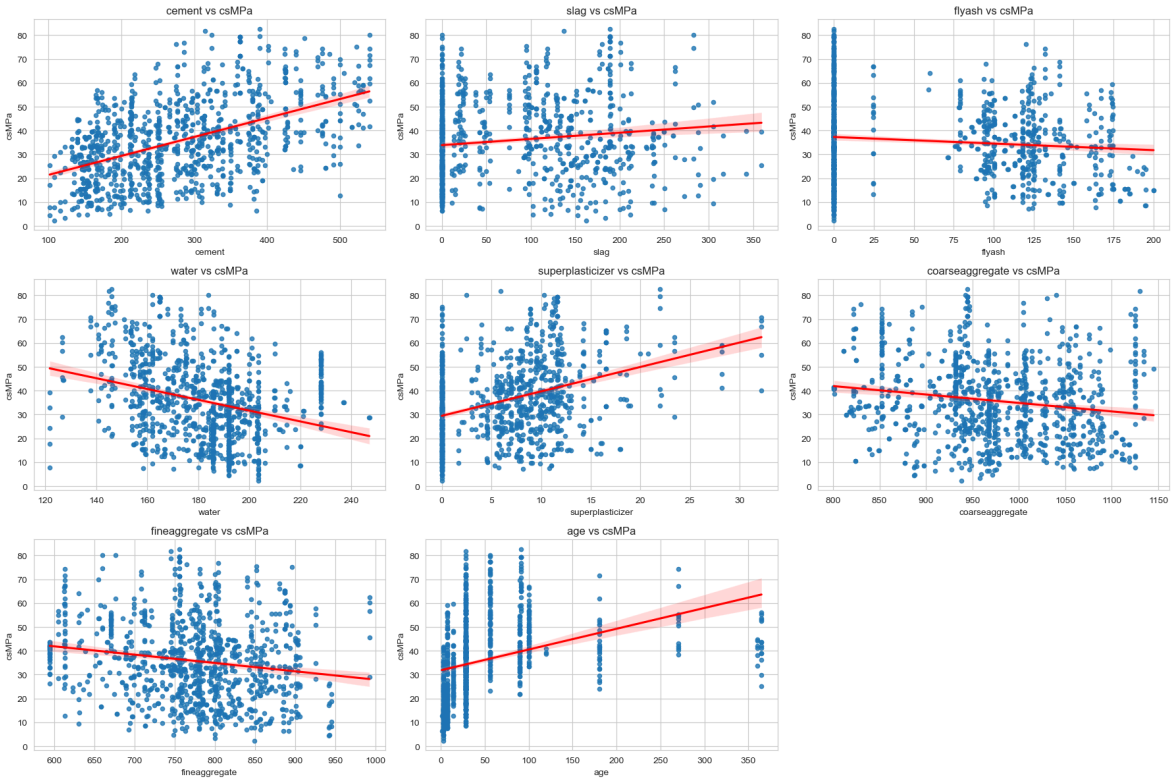
相关性分析

相关性分析结果表明，水泥含量（cement）与抗压强度呈正相关，而水（water）与抗压强度呈负相关，不同材料对强度的影响程度存在差异。

In [20]:
```
corr = data.corr()
plt.figure(figsize=(8,6))
sns.heatmap(corr,annot=True,cmap="coolwarm",fmt=".2f")
```

```
plt.title("Correlation Matrix")
plt.show()
def plot_scatter_with_reg(data, target="csMPa"):
    sns.set_style("whitegrid")
    features = data.columns.drop(target)
    fig, axes = plt.subplots(3, 3, figsize=(18, 12))
    axes = axes.flatten()
    for i, col in enumerate(features):
        sns.regplot(data=data,x=col,y=target,ax=axes[i],scatter_kws={"s": 20}, l
        axes[i].set_title(f"{col} vs {target}")
    fig.delaxes(axes[-1])
    plt.tight_layout()
    plt.show()
plot_scatter_with_reg(data)
```





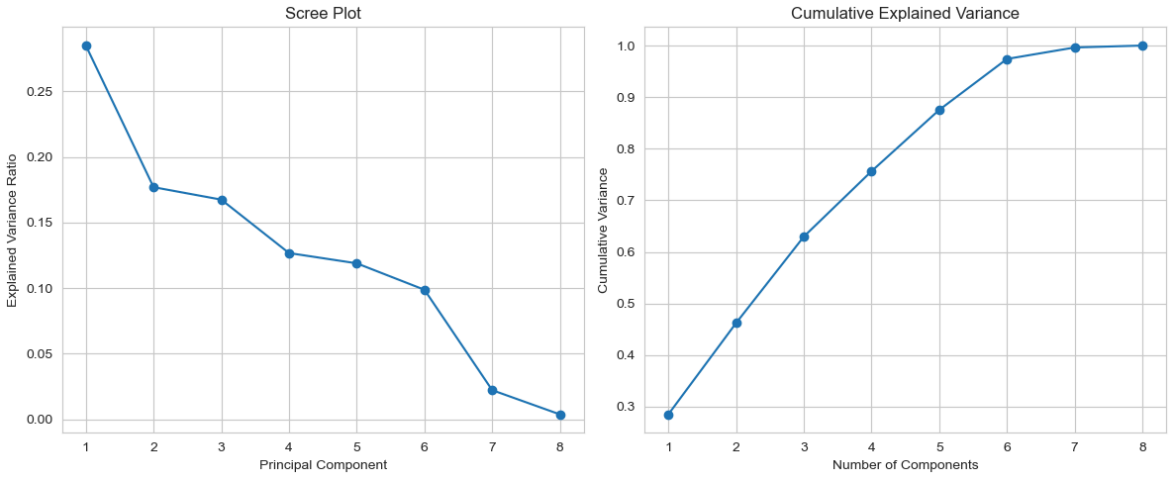
主成分分析

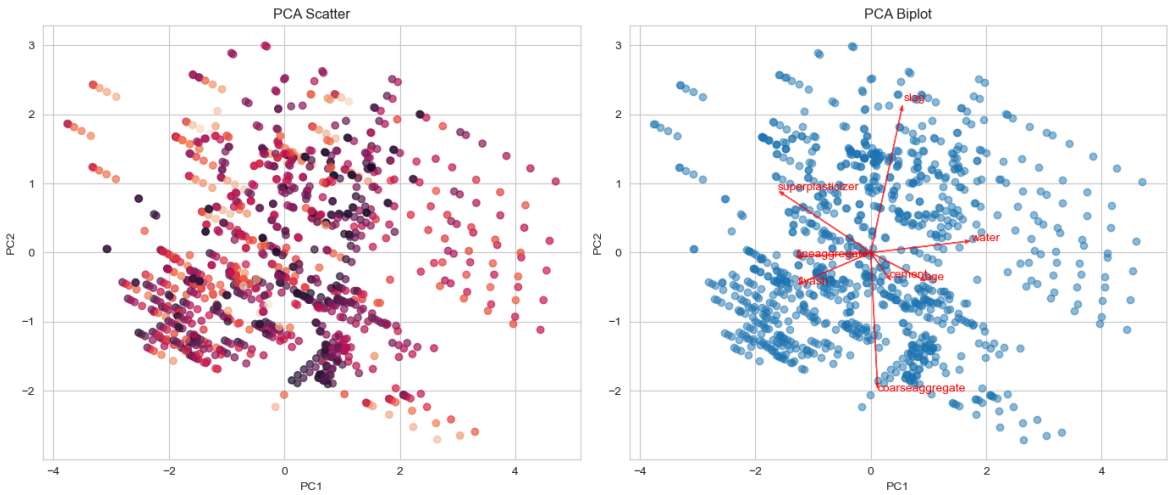
PCA通过对协方差矩阵进行特征分解，将原始特征转换为新的正交特征（主成分），以实现降维。前6个主成分已能够保留约97%以上的信息，因此选取6个主成分进行后续建模

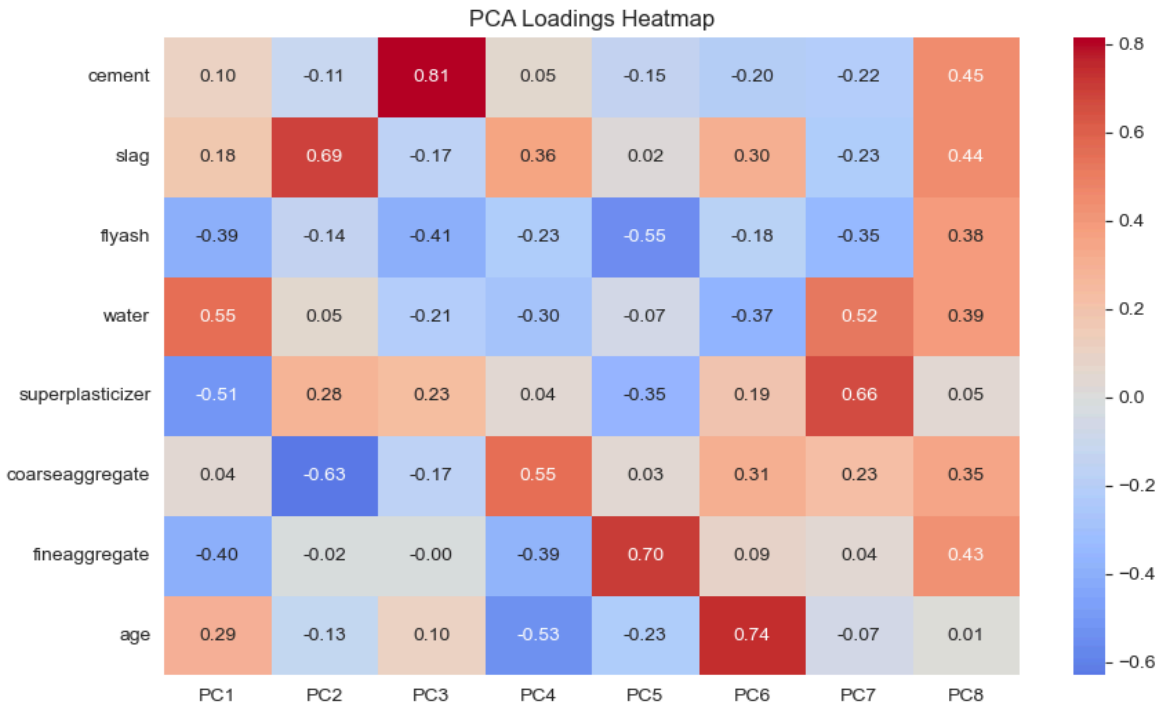
In [23]:
```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
# 分离特征
X = data.drop("csMPa", axis=1)

# 标准化
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

# 查看每个主成分解释的方差比例
explained_var = pca.explained_variance_ratio_
cum_explained_var = np.cumsum(explained_var)

print("各主成分方差贡献率：\n", explained_var)
print("累计方差贡献率：\n", cum_explained_var)
#查看每个主成分贡献了多少信息，以及累计贡献了多少信息
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
# 左图：Scree Plot
axes[0].plot(range(1, len(explained_var) + 1), explained_var, marker='o')
axes[0].set_title("Scree Plot")
axes[0].set_xlabel("Principal Component")
axes[0].set_ylabel("Explained Variance Ratio")
# 右图：累计方差
axes[1].plot(range(1, len(cum_explained_var) + 1), cum_explained_var, marker='o'
```

```
axes[1].set_title("Cumulative Explained Variance")
axes[1].set_xlabel("Number of Components")
axes[1].set_ylabel("Cumulative Variance")
plt.tight_layout()
plt.show()
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
#查看前两个主成分的散点图和双标图
# 左: PCA散点图
scatter = axes[0].scatter(X_pca[:, 0],X_pca[:, 1],c=data["csMPa"],alpha=0.7)
axes[0].set_title("PCA Scatter")
axes[0].set_xlabel("PC1")
axes[0].set_ylabel("PC2")

# 右: Biplot
axes[1].scatter(X_pca[:, 0], X_pca[:, 1], alpha=0.5)
for i, feature in enumerate(X.columns):
    axes[1].arrow(0, 0,loadings.iloc[i, 0] * 3,loadings.iloc[i, 1] * 3,color='re
    axes[1].text(loadings.iloc[i, 0] * 3.2,loadings.iloc[i, 1] * 3.2,feature,col
axes[1].set_title("PCA Biplot")
axes[1].set_xlabel("PC1")
axes[1].set_ylabel("PC2")

plt.tight_layout()
plt.show()
#查看每个主成分的载荷（
loadings = pd.DataFrame(pca.components_.T,index=X.columns,columns=[f"PC{i+1}" fo
plt.figure(figsize=(10, 6))
sns.heatmap(loadings, annot=True, cmap="coolwarm", center=0, fmt=".2f")
plt.title("PCA Loadings Heatmap")
plt.show()
```

```
各主成分方差贡献率:
 [0.28501242 0.17700935 0.16750988 0.12676986 0.1189385  0.09877443
 0.02223021 0.00375535]
累计方差贡献率:
 [0.28501242 0.46202177 0.62953165 0.75630151 0.87524001 0.97401444
 0.99624465 1.        ]
```







线性回归

In [35]:
```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

X = data.drop("csMPa", axis=1)
y = data["csMPa"]

# 按顺序划分数据（80%训练，20%测试）
split_idx = int(0.8 * len(data))
X_train = X.iloc[:split_idx]
X_test  = X.iloc[split_idx:]

y_train = y.iloc[:split_idx]
y_test  = y.iloc[split_idx:]

#  数据预处理（标准化）
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

```python
#  PCA降维（6个主成分）
pca = PCA(n_components=6)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca  = pca.transform(X_test_scaled)

# 模型1：原始特征回归
model_raw = LinearRegression()
model_raw.fit(X_train_scaled, y_train)

y_pred_train_raw = model_raw.predict(X_train_scaled)
y_pred_test_raw  = model_raw.predict(X_test_scaled)

#  模型2：PCA回归
model_pca = LinearRegression()
model_pca.fit(X_train_pca, y_train)

y_pred_train_pca = model_pca.predict(X_train_pca)
y_pred_test_pca  = model_pca.predict(X_test_pca)

# 评价指标
print("===== 原始特征模型 =====")
print("Train R2:", r2_score(y_train, y_pred_train_raw))
print("Test R2 :", r2_score(y_test, y_pred_test_raw))
print("Train MSE:", mean_squared_error(y_train, y_pred_train_raw))
print("Test MSE :", mean_squared_error(y_test, y_pred_test_raw))

print("\n===== PCA模型 =====")
print("Train R2:", r2_score(y_train, y_pred_train_pca))
print("Test R2 :", r2_score(y_test, y_pred_test_pca))
print("Train MSE:", mean_squared_error(y_train, y_pred_train_pca))
print("Test MSE :", mean_squared_error(y_test, y_pred_test_pca))


# 残差
residual_raw = y_test - y_pred_test_raw
residual_pca = y_test - y_pred_test_pca

# 绝对误差
abs_error_raw = np.abs(residual_raw)
abs_error_pca = np.abs(residual_pca)

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

#  Model Comparison
axes[0,0].scatter(y_test, y_pred_test_raw, alpha=0.6, label="Raw")
axes[0,0].scatter(y_test, y_pred_test_pca, alpha=0.6, label="PCA")

axes[0,0].plot([y_test.min(), y_test.max()],
               [y_test.min(), y_test.max()],
               'r--', label="Ideal")

axes[0,0].set_title("Model Comparison",fontsize=14, weight="bold")
axes[0,0].set_xlabel("True csMPa")
axes[0,0].set_ylabel("Predicted csMPa")
axes[0,0].legend()


# Residual Scatter
axes[0,1].scatter(y_test, residual_raw, alpha=0.7, label="Raw")
axes[0,1].scatter(y_test, residual_pca, alpha=0.7, label="PCA")
```

```
axes[0,1].axhline(0, linestyle='--')
axes[0,1].set_title("Residual vs True Value",fontsize=14, weight="bold")
axes[0,1].set_xlabel("True csMPa")
axes[0,1].set_ylabel("Residual")
axes[0,1].legend()


# Residual Distribution
sns.histplot(residual_raw, kde=True, ax=axes[1,0], color="blue", label="Raw", st
sns.histplot(residual_pca, kde=True, ax=axes[1,0], color="orange", label="PCA",
axes[1,0].set_title("Residual Distribution",fontsize=14, weight="bold")
axes[1,0].legend()


# Boxplot
error_df = pd.DataFrame({
    "Absolute Error": np.concatenate([abs_error_raw, abs_error_pca]),
    "Model": ["Raw"] * len(abs_error_raw) + ["PCA"] * len(abs_error_pca)
})
sns.set(style="whitegrid", context="talk")
sns.boxplot(data=error_df,x="Model",y="Absolute Error",hue="Model",palette=["#4C
axes[1,1].set_title("Absolute Error Comparison", fontsize=14, weight="bold")
axes[1,1].set_ylabel("|Error|")

plt.tight_layout()
plt.show()
```

```
===== 原始特征模型 =====
Train R2: 0.6181905004215971
Test R2 : 0.5037418272955774
Train MSE: 117.45245610387944
Test MSE : 75.74392466040452

===== PCA模型 =====
Train R2: 0.5934082901105865
Test R2 : 0.1612608795699041
Train MSE: 125.07597377938289
Test MSE : 128.01681915157096
```

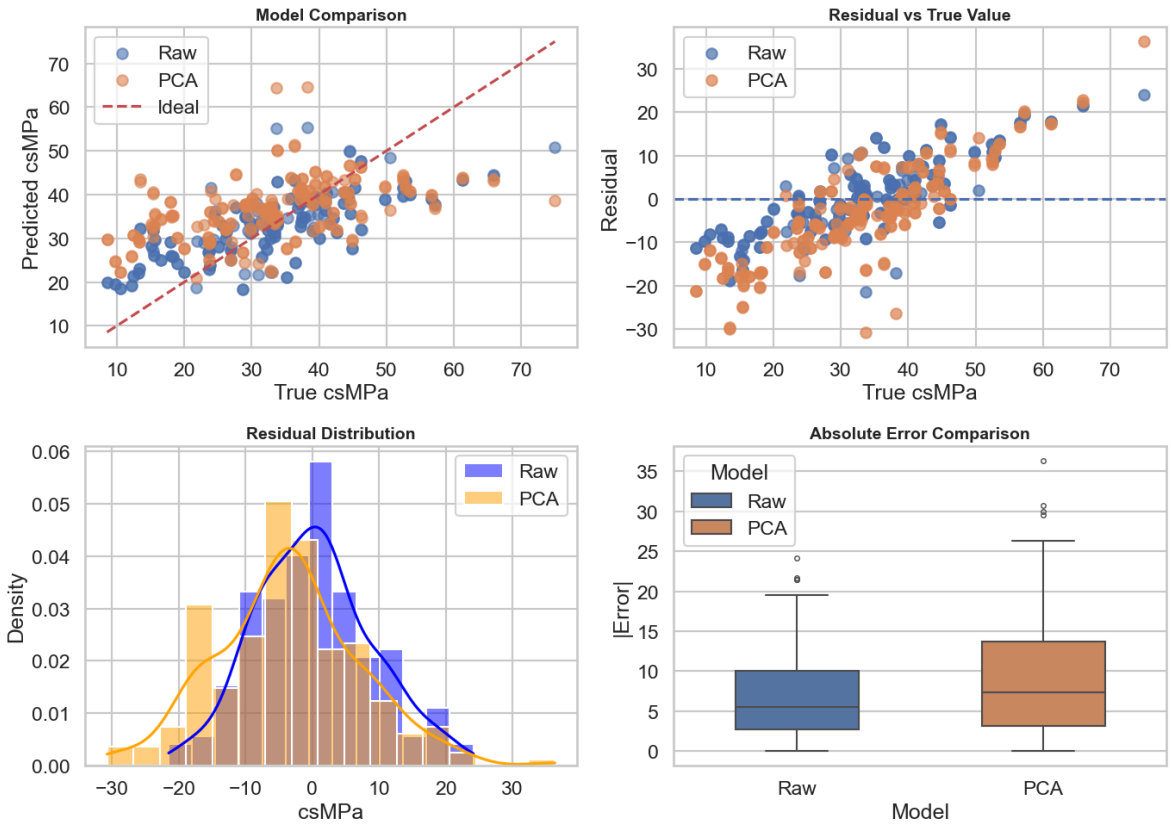


图1：原始模型的预测结果更接近理想对角线，而PCA模型存在更明显偏离，说明原始模型拟合效果更好

图2：残差大致围绕0分布，但在高强度区域误差增大，说明模型在高值区域预测能力较弱，存在一定的异方差现象。

图3：原始模型残差分布更加集中，而PCA模型分布更分散，说明其误差更大且稳定性较差

图4：从箱线图可以看出，原始模型的绝对误差中位数更低，且分布更集中，而PCA模型误差更大且存在更多离群点，说明原始特征模型在预测精度和稳定性方面均优于PCA模型。

本实验通过对比原始特征模型与PCA降维模型，发现尽管PCA能够有效降低特征维度并减少多重共线性，但由于其不考虑目标变量信息，在降维过程中可能丢失关键特征信息，导致预测性能下降。因此，在本数据集上，原始特征线性回归模型表现优于PCA模型